

Self-Grading Agent

Complete Technical Interview Master Guide & Architecture Reference

60-Second Interview Pitch:

"Standard RAG systems suffer from sycophancy and uncalibrated overconfidence—when an LLM hallucinates, asking it to verify its own text creates confirmation bias. I designed and built a **Self-Grading Agent** with a strictly decoupled architecture: generation is powered by Google Gemini (extracting structured atomic claims), verification is performed independently by a local Cross-Encoder NLI model (MiniLM2-L6-H768) alongside deterministic symbolic validators, and a hard decision engine assigns calibrated confidence labels (High, Low, or I don't know). The system guarantees zero dangerous overconfidence on enterprise benchmark tests."

1. System Architecture & 4-Stage Agent Pipeline

The core principle is the **strict separation of Generation and Verification**. The pipeline consists of 4 distinct, independently testable stages:

Stage	Component & Tech Stack	Key Responsibilities & Operations
Stage 1: Retrieval	Hybrid Search - FAISS (BGE-base-en-v1.5) - BM25Okapi (Lexical)	Ingests markdown corpus; encodes 768-dim dense embeddings + sparse BM25 indices. Performs Reciprocal Rank Fusion (60% Dense + 40% BM25) to capture both semantic concepts and exact keyword numbers (\$49, dates, codes).
Stage 2: Generation	Google Gemini - Gemini 2.5 / 3.8 Flash - Pydantic Schema	Consumes top retrieved passages under strict zero-shot constraints. Generates a grounded draft answer and decomposes it into discrete atomic factual claims (Pydantic DraftAnswer). Includes automated fallback cascade for 429/503 spikes.
Stage 3: Verification	Local NLI + Symbolic - Cross-Encoder MiniLM2 - Entity / Num Guards	Extracts section-grounded premise lines from passages. Cross-Encoder computes softmax probabilities [Entailment, Neutral, Contradiction] for each atomic claim. Regex symbolic guards cross-check dates, numbers, currency, and conditionals.
Stage 4: Decision	Deterministic Rules - Rule Hierarchy 3 Standard Labels	Applies hard deterministic precedence: Abstain if missing evidence/abstain flag; High confidence if Support ≥ 0.82 and all guards pass; Low confidence for partial support/qualifiers. Produces auditable one-line justification.

2. Deep Dive: Model Selection & Technical Rationale

• Why BAAI/bge-base-en-v1.5 for Dense Embeddings?

BGE-base is consistently a top performer on the Massive Text Embedding Benchmark (MTEB). It produces high-density 768-dimensional embeddings that excel in capturing sentence-level semantic relatedness. It is lightweight, fast to compute on CPU, and normalized with inner-product cosine similarity in FAISS.

• Why Hybrid Retrieval (FAISS + BM25) instead of pure Vector Search?

Pure dense vectors frequently fail on subtle keyword traps (e.g., confusing \$49/user with \$79/workspace, or misidentifying version numbers like v2.4 vs v2.5). BM25 guarantees exact keyword matching for product names, pricing digits, and dates, while FAISS handles semantic paraphrases. Combining them via score fusion eliminates blind spots.

• Why a Local Cross-Encoder NLI Model (nli-MiniLM2-L6-H768)?

1. **Full Cross-Attention:** Bi-encoders encode premise and hypothesis separately (cosine similarity), which measures topical similarity rather than logical truth. Cross-encoders perform joint cross-attention over [CLS] Premise [SEP] Hypothesis [SEP], detecting true entailment vs contradiction.
2. **Zero Prompt Bias:** It eliminates LLM self-confirmation bias.
3. **Latency & Cost:** Runs locally in <15ms on CPU with zero network API costs, quota constraints, or outages.

• Why Decoupled Verification instead of LLM-as-a-Judge?

LLM-as-a-judge calls are 2x-3x more expensive, introduce double the API latency, and suffer from prompt-induced hallucination cascades. A local NLI model provides deterministic, mathematical probability distributions that are auditable and reproducible.

3. Comprehensive File-by-File Codebase Map

A guided breakdown of every file in the repository for architectural review:

File Path	Module / Role	Key Logic & Purpose
app/config.py	Configuration	Pydantic BaseSettings loading environment variables (GEMINI_API_KEY, model names, top-k, weights, index paths).
app/models/schemas.py	Data Contracts	Pydantic models for RetrievedPassage, DraftAnswer (claims, abstain), ClaimCheck, and FinalAnswer.
app/models/verification.py	Verification Schema	Data model VerificationResult storing support scores, contradiction scores, entity/temporal/numerical match booleans.
app/ingestion/loader.py	Doc Ingestion	Parses markdown files in data/knowledge_base/, extracting metadata headers (Doc ID, Title, Source) and structured text chunks.
app/ingestion/indexer.py	Indexing Engine	Computes BGE dense embeddings, builds L2-normalized FAISS IndexFlatIP, and builds BM25Okapi pickled index.
app/retrieval/retriever.py	Hybrid Retriever	HybridRetriever class executing dense FAISS search + sparse BM25 search, fusing scores via normalized weights (0.6 / 0.4).
app/generation/generator.py	Generation Engine	GeminiGenerator enforcing zero-shot grounded prompts, structured JSON schema response, and automated model fallback cascade.
app/verification/verifier.py	NLI Verifier	NliVerifier executing local Cross-Encoder batch inference, premise line extraction, contradiction detection, and regex entity guards.
app/decision/confidence.py	Decision Engine	ConfidenceDecision enforcing deterministic priority rules: Abstain -> High Confidence (Score >= 0.82) -> Low Confidence.
app/pipeline/agent.py	Agent Orchestrator	SelfGradingAgent facade connecting Retrieval -> Generation -> Verification -> Decision into a clean run(question) API.
app/main.py	Streamlit Web UI	Interactive browser application featuring Q&A; input, confidence badges, justification explanations, NLI inspector, and KB browser.

run.py	CLI Master Runner	Command-line runner supporting the 4-level live demonstration, custom question queries, interactive REPL shell (-i), and UI launcher.
scripts/build_index.py	Build Script	One-command utility to index/re-index knowledge base documents into indexes/faiss and indexes/bm25.
scripts/run_evaluation.py	Evaluation Harness	Executes the 25 benchmark questions, computes metrics, and outputs CSV/JSON reports and the confusion matrix plot.
POST_MORTEM.md	Engineering Report	Architectural post-mortem analyzing decoupling, trade-offs, failure modes at \$10^5\$ scale, and production hardening roadmap.
FAILURE_ANALYSIS.md	Failure Analysis	4 granular failure case studies diagnosing root causes across retrieval, generation, verification, and thresholding with actionable fixes.

4. Scenario-Based Technical Interview Q&A;

Scenario 1: How does the agent handle Trap Questions (e.g. Swapped Price / Product)?

Question Asked: 'Does NovaSearch cost \$79 per workspace per month?'

System Behavior:

1. *Hybrid Retrieval:* BM25 and FAISS fetch 01_nexatech.md (NovaSearch is \$49/user) and 08_pricing_matrix.md (FlowPilot is \$79/workspace).
2. *Generation:* The LLM detects the discrepancy and generates an explicit correction stating NovaSearch is \$49/user while FlowPilot is \$79/workspace.
3. *Atomic Decomposition:* It extracts positive verifiable claims for both products.
4. *NLI Verification:* Cross-Encoder verifies both factual propositions against the retrieved matrix (Entailment = 0.99).
5. *Outcome:* **High confidence** because the correction is fully supported by the knowledge base.

Scenario 2: How do you prevent hallucinations on Unanswerable Queries?

Question Asked: 'What was NexaTech Solutions' total revenue in 2026?'

System Behavior:

1. The knowledge base contains no 2026 financial records.
2. The generator recognizes missing data and outputs abstain=True with no unsupported claims.
3. The decision engine encounters the abstention signal and immediately routes to: **'I don't know — the documents do not contain enough evidence to answer.'**

Scenario 3: How do you handle Ambiguous or Conditional Questions?

Question Asked: 'Does NexaTech provide 24/7 customer support?'

System Behavior:

1. Retrieved evidence states standard support is M-F 09:00-18:00, while 24/7 support is conditional on Premium Support tier.
2. The system generates a nuanced conditional answer.
3. If a claim omits the condition qualifier, the verifier flags condition_match=False, forcing a calibrated **Low confidence** outcome.

Scenario 4: What is Dangerous Overconfidence and how is it measured?

Definition: A critical safety metric where the agent delivers a *High confidence* label for an answer that is factually wrong.

Formula: Dangerous Overconfidence = (Wrong Answers with High Confidence) / (Total High Confidence Predictions).

Our Result: 0 cases (0.0%) across the entire 25-question benchmark dataset.

Scenario 5: What breaks at Enterprise Scale (100,000+ Documents) and how do you scale it?

- 1. **Verification Latency Explosion:** Evaluating M atomic claims against K passages creates O(M x K) Cross-Encoder calls. *Fix:* Implement Hierarchical Verification (Bi-Encoder ColBERT fast filter to prune 90% of pairs, followed by Cross-Encoder only on top-2 passages).
- 2. **Multi-Hop Premise Fragmentation:** When facts span multiple documents, single-chunk NLI classifies combined deductions as neutral. *Fix:* Multi-passage knowledge graph synthesis.
- 3. **Threshold Drift:** *Fix:* Replace static empirical thresholds with Conformal Risk Control to guarantee P(Error | High Confidence) <= 1% statistically.

5. Live Demonstration Quick-Reference Cheat Sheet

Category	Test Question to Type	Expected Label & Key Signal
1. Answerable	'When was NovaSearch launched?'	High confidence (Entailment: 0.99, Answer: March 2025)
2. Unanswerable	'What was NexaTech Solutions' total revenue in 2026?'	I don't know (Abstention flag, Evidence Sufficient: No)
3. Ambiguous	'Does NexaTech provide 24/7 customer support?'	High / Low confidence (Details Standard M-F vs Premium 24/7 tier)
4. Trap Question	'Does NovaSearch cost \$79 per workspace per month?'	High confidence (Corrects trap: NovaSearch is \$49/user, FlowPilot is \$79/workspace)

Summary for the Interviewer: This system is production-ready, fully tested (25/25 pytest passing), dockerizable, and strictly adheres to the calibrated confidence criteria.